MLOPS LABORATORY

PriceOptima

An end-to-end system for dynamic retail pricing

This report states the aim, the laboratory objectives, the data split, the training workflow, the serving architecture, the experiment record, and the deployment path. The same material is in the slide deck `docs/PriceOptima_Presentation.pptx`.

Repository	shreyanshhhhhhhh/MLOps_FinaL
Branch	main
Dataset	Kaggle Retail Price Optimization
Cloud	AWS free tier, eu-north-1 (Stockholm)
Date	30 October 2026

Contents

1. Aim and objectives
2. Problem, dataset, and the chronological split
3. Workflow and the three control files
4. Architecture and AWS deployment
5. Experiments, model versions, and CI/CD
6. API, MLOps map, and how to reproduce

1. Aim

Build a system that predicts how many units a retail product will sell in a month, then chooses a price that maximises profit, and run that system the way a production model is run: versioned data, tracked experiments, one promoted model, a service that uses the same features as training, and a deployment that can roll back.

A retailer sets a price before the month begins. The useful prediction is the quantity that price will sell. Profit at a candidate price is predicted quantity times (price minus unit cost). The service scores a grid of prices and keeps the best one. If unit cost is omitted, the score is revenue.

2. Objectives

1. Turn the Kaggle retail-price table into a reproducible pipeline whose settings live in one file.
2. Train several model families, record every run, and promote the run with the lowest test error.
3. Hold out future months, so the test score uses months the model was never fit on.
4. Serve the promoted model over HTTP, and compute a recommended price from the same model.
5. Keep the training code and the serving code on one feature implementation.
6. Ship the service in a container that downloads the model at start. The pickle stays out of the image.
7. Check every change with lint, tests, a DVC lock check, and an image build.
8. Deploy the container on AWS free-tier sizes: one private S3 bucket and one `t3.micro` instance.

These line up with the laboratory outcomes: cloud and MLOps fundamentals, pipeline and CI/CD, tracking and optimisation, and documentation plus deployment.

3. Problem and dataset

The dataset is `retail_price.csv`: 676 product-months, 30 columns, January 2017 through August 2018. Each row is one product in one month. The target column is `qty`.

The split is chronological. `params.yaml` sets `test_size: 0.2` and `strategy: chronological`. The file has 20 distinct months. The last four months are the entire test set. A month is never cut in half. The test RMSE is computed once, on those four months. The live API does not re-score a fraction of the table every month.

Part	Months	Rows
Train	16 months, January 2017 – April 2018	516
Test	4 months, May 2018 – August 2018	160

Outlier fences are Tukey intervals fit on the training months only, then applied to both sides. Rows are clipped, not dropped, so the product-month panel stays complete.

4. Workflow

One command, `dvc repro`, runs the five stages. A stage whose inputs did not change is skipped. Git stores the code and the lock. DVC stores the CSV and the model by hash. Two machines with the same lock get the same bytes.

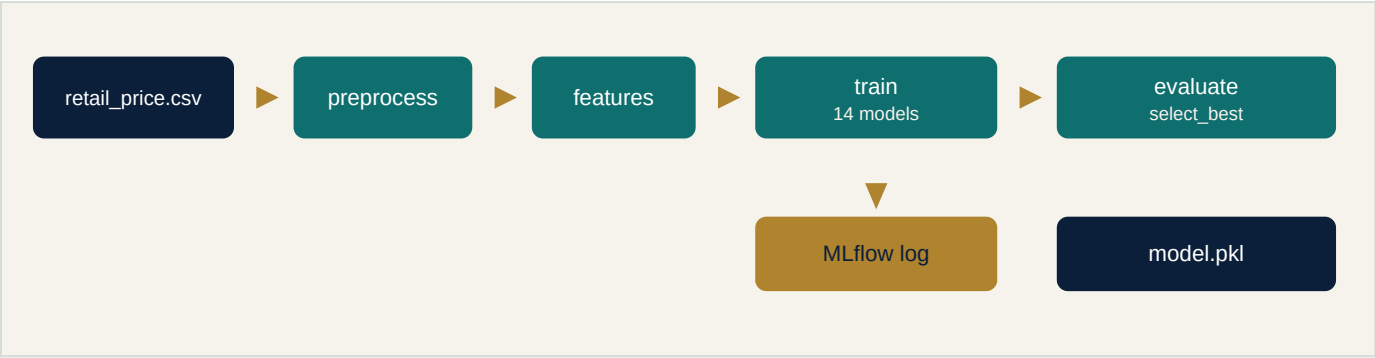


Figure 1. Training workflow. `dvc repro` runs preprocess, features, train, evaluate, and select_best. Training writes one MLflow run per model and select_best writes the promoted pickle.

Stage	Program	What it writes
preprocess	`src/preprocess.py`	`train.csv`, `test.csv`, `preprocess_meta.json`
features	`src/features.py`	feature tables and `preprocessor.joblib`
train	`src/train.py`	`train_runs.json` and an MLflow run for each model
evaluate	`src/evaluate.py`	`metrics.json` and residual plots
select_best	`src/select_best.py`	`model.pkl` and `model_meta.json`

5. The three control files

params.yaml is the settings. Seed 42, the 20% chronological split, the date format `%d-%m-%Y`, the target `qty`, the feature list, and the 14-model grid. Change a value here, then `dvc repro` reruns only the stages that depend on it.

dvc.yaml is the recipe. It names the command, the input files, the parameters, and the output files for each stage. It does not contain the CSV or the pickle.

dvc.lock is the proof. After a run, DVC writes the MD5 of the code, of the parameter values, and of every output. CI refuses the branch unless `dvc status` and `dvc repro --dry` both say the pipeline is up to date.

6. Architecture and deployment

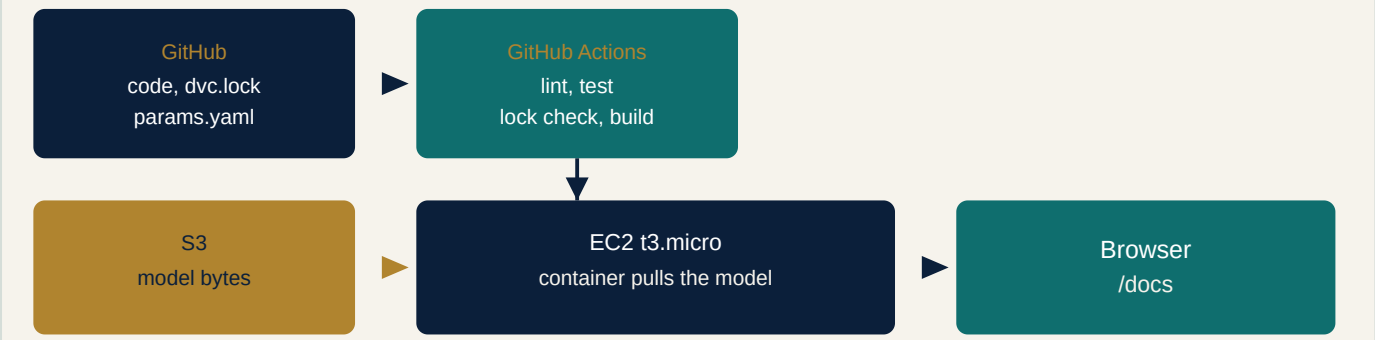


Figure 2. Code is checked in GitHub Actions, then the container runs on EC2. The model bytes move from S3 into that container. The browser talks only to the API.

Piece	Where it is
Code and lock	GitHub, branch `main`
CSV and model hashes	S3 bucket `priceoptima-sagar-2026`, prefix `priceoptima`, region `eu-north-1`
Running API	EC2 instance `priceoptima`, type `t3.micro`, Ubuntu, port 8000
Credentials on the server	IAM role `PriceOptimaEc2`, read-only on that prefix
Laptop upload key	IAM user `priceoptima-laptop`, used only for `dvc push`. It is not stored in Git

> `/health` is 200 as soon as the process is up. `/ready` stays 503 until `model.pkl`, `preprocessor.joblib`, and `model_meta.json` have been loaded. Serving calls `add_engineered_features` and `transform_frame` from `src/features.py`. Those are the same functions the features stage runs.

The public page is `http://<public-ip>:8000/docs`. Port 8000 must be open in the security group. The public IPv4 changes if the instance is stopped and started. Stop the instance when it is not being demonstrated.

7. Experiment results

Fourteen runs are logged in the MLflow experiment `priceoptima-demand`. The selection metric is test RMSE. Lower is better. The best experiment is gradient boosting: 100 trees, learning rate 0.05, depth 3, test RMSE 7.53, MAE 5.43, R^2 0.50.

Family	Best setting	Test RMSE	Test MAE	Test R^2
Gradient boosting	100 trees, learning rate 0.05, depth 3	7.53	5.43	0.50
Random forest	100 trees, depth 16	7.76	5.66	0.47
LightGBM	100 trees, learning rate 0.1, 31 leaves	7.83	5.59	0.46
Ridge	alpha 10	10.76	9.25	−0.02
Linear regression	fit intercept	10.99	9.47	−0.06

Two LightGBM settings, 31 leaves and 63 leaves, land on the same test RMSE. The training panel has 516 rows and LightGBM stops splitting before it can use 63 leaves. Both settings stay in the log, because the grid is the experiment record.

For the sample bed product (`bed_bath_table`, May 2018, price 39.99) the recorded sales were 19 units. Gradient boosting predicts about 7.91.

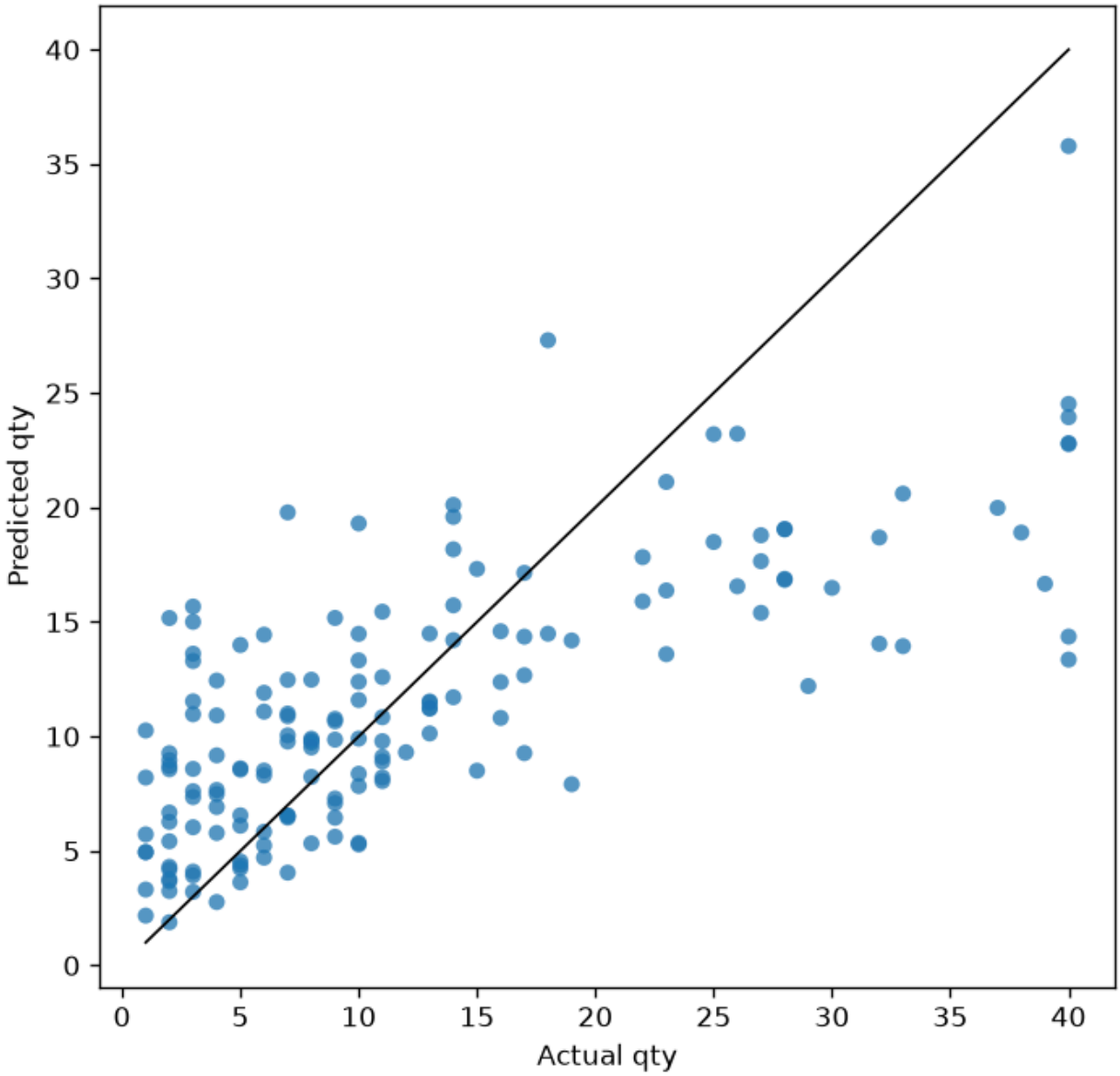


Figure 3. Gradient boosting on the four test months: predicted quantity against actual quantity. Points on the diagonal would be perfect predictions.

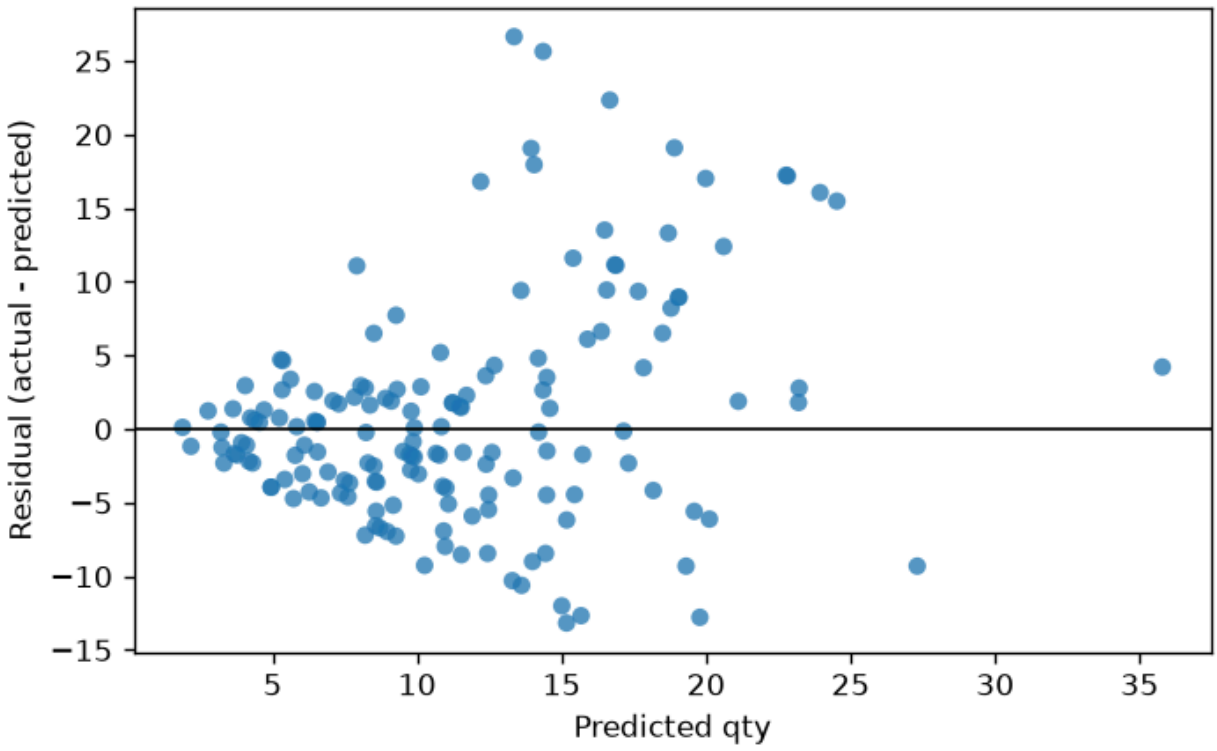


Figure 4. Residuals for the same gradient boosting run. A residual is actual quantity minus predicted quantity.

8. Two model versions

Tag	Model	Test RMSE	Role
`model-v1`	Gradient boosting	7.53	Best experiment. Pulled when `MODEL_VERSION=model-v1`.
`model-v2`	Ridge, alpha 10	10.76	The lock the current image pulls. Kept so a version swap is visible.

`/optimize-price` scores every price from `min_price` to `max_price`. The response includes `best_price`, expected demand, expected profit, and the elasticity around the chosen price.

> In the viva, say which file is loaded. `/model-info` reports the card that was pulled. With the current lock that card is ridge (`model-v2`). Gradient boosting is the best experiment and is available as `model-v1`.

9. CI/CD

`.github/workflows/ci.yml` runs four jobs, in order. A failure stops the rest.

Job	What it checks
lint	Ruff on `src`, `app`, and `tests`
test	Pytest, with a coverage report uploaded as an artifact
pipeline-check	`dvc pull`, then `dvc status` and `dvc repro --dry` must say the lock is up to date
build	`docker build`. The image is not pushed from this job

If `AWS_ROLE_ARN` or `AWS_ACCESS_KEY_ID` is set, the lock check pulls from S3. If neither secret is set, it pulls the committed `dvc-storage/` directory. That is why CI can be green with no AWS account. Adding a wrong key

switches the job to S3 and the pull fails.

`.github/workflows/cd.yml` is the release path. It runs only after CI succeeds on a push to `main` or a `v*` tag. It smoke-tests the container, fails the release on a critical vulnerability, pushes the image, and connects to EC2. `scripts/deploy.sh` records the previous image, starts the new one, and waits up to 120 seconds for `/ready`. If that check fails, it starts the previous image. The container is started with `--restart unless-stopped`, so a reboot brings it back.

The demonstration on 29 October 2026 built the image on the instance and started it with `scripts/deploy.sh`. The instance check of `/ready` returned `{"status":"ready"}`.

10. API used in the demonstration

Endpoint	Role
`GET /health`	Process is up
`GET /ready`	Model files are loaded
`GET /model-info`	Name, test RMSE, git commit, data hash
`POST /predict-demand`	Quantity at a given price
`POST /optimize-price`	Best price on a grid
`GET /metrics`	Request counter in Prometheus text
`GET /docs`	Swagger UI, the screen used in the viva

11. What each MLOps idea means here

Idea	Where it is
Single configuration	`params.yaml`
Reproducible pipeline	`dvc.yaml` and `dvc.lock`
Data versioning	`data/raw/retail_price.csv.dvc` and the S3 remote
Experiment tracking	MLflow experiment `priceoptima-demand`
Model registry	`model.pkl`, `model_meta.json`, tags `model-v1` and `model-v2`
One feature implementation	`src/features.py`, imported by `app/model_loader.py`
Packaging	`Dockerfile`. The model is not copied into the image
Continuous integration	`.github/workflows/ci.yml`
Deployment and rollback	`scripts/deploy.sh` on EC2
Monitoring in this lab	`/ready`, `/metrics`, and `scripts/healthcheck.sh`

> Monitoring here is a readiness check and a request counter. It is a liveness and load signal for the viva. It is separate from a data-drift detector, and that limit should be said plainly.

12. How to reproduce

On a laptop, from a clone of `main`:

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
dvc pull
pytest
```

The API without Docker:

```
pip install -r requirements-serve.txt
dvc pull
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Open `http://127.0.0.1:8000/docs`. On Windows the Python launcher is `py`, and the activate command is `.venv\Scripts\activate`.

13. Conclusion

PriceOptima is a small pricing service with the MLOps path made visible. The pipeline is deterministic given `params.yaml` and the lock. The experiment record names the winner and also keeps a second model so a rollback changes the prediction. The image and the model are versioned separately. The running copy on EC2 pulls the model from S3 with a read-only role, and `/ready` is the gate that says the model is actually loaded.